Data Analysis Report

Hanna Oh

2026-04-16

Busiest Airport Analysis

This section examines passenger traffic for six selected airports from 2020 to 2025. The data were scraped from Wikipedia, combined across years, and cleaned by separating airport codes and converting passenger values into numeric form.

Table 1: Total Annual Passengers for Six Selected Airports, 2020-2025

Airport	IATA	2020	2021	2022	2023	2024	2025
Hartsfield–Jackson Atlanta International Airport	ATL	61.2	76.4	23.8	11.7	3.3	1.6
Frankfurt Airport	FRA	73.4	32.2	97.2	21.3	3.7	2.6
Heathrow Airport	LHR	72.7		217.7	28.5	5.9	0.7
O'Hare International Airport	ORD	63.5	75.1	26.5	8.1	8.3	6.0
Phoenix Sky Harbor International Airport	PHX	52.6	77.2	14.3	9.6	17.9	1.3
Beijing Daxing International Airport	PKX		55.7			25.5	8.4

Note: Blank cells indicate years in which passenger data were not available in the scraped table.

Table 1 shows the total annual passenger counts for each airport across the six-year period. The table highlights differences in passenger volume across airports, as well as missing values for certain years when an airport did not appear in the ranking.

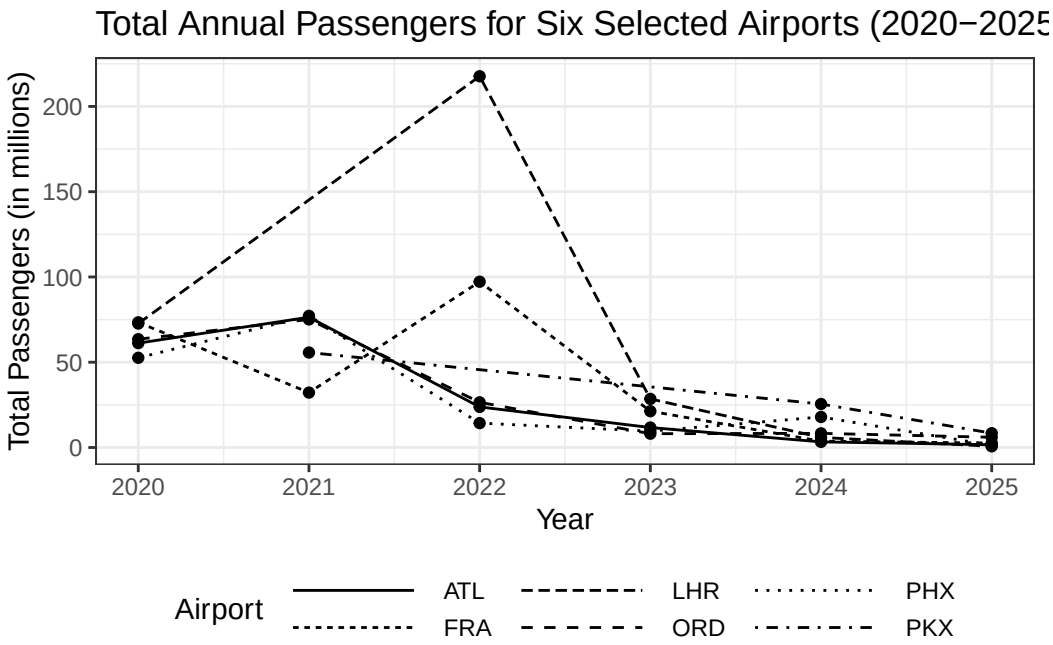

Figure 1: Total Annual Passengers for Six Selected Airports (2020–2025)

Figure 1 illustrates how passenger traffic changed over time for each airport. Passenger traffic patterns differed across airports, with some airports showing sharper fluctuations than others during the 2020–2025 period.

Monte Carlo Numerical Integration

This section uses Monte Carlo numerical integration to estimate the area under a Weibull density curve on the interval from 0 to 4. Random points are generated inside a rectangle, and the proportion of points on or below the curve is used to estimate the integral.

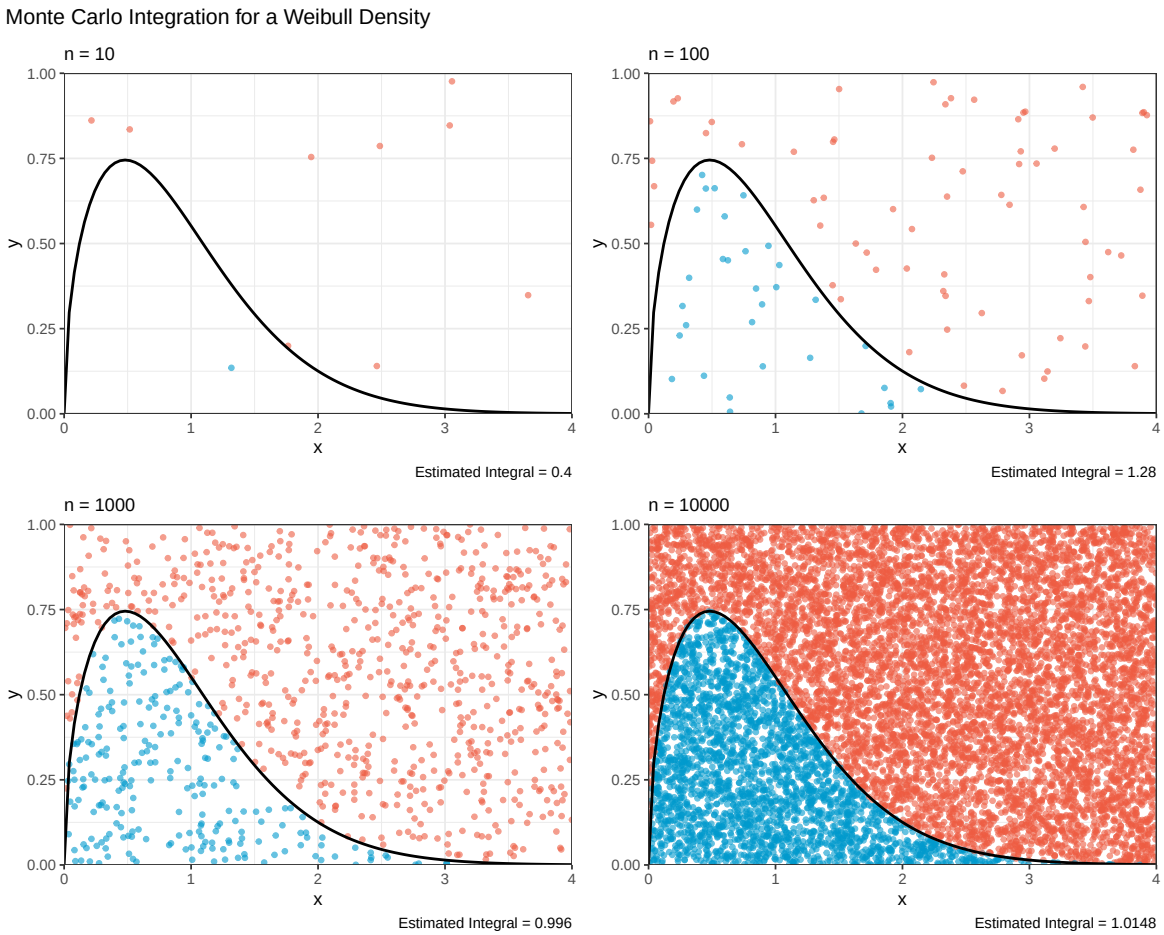

Figure 2: Monte Carlo Integration for a Weibull Density

Figure 2 shows four Monte Carlo simulations using different sample sizes: 10, 100, 1000, and 10000. When the sample size is small, the points are sparse and the approximation is less stable. As the sample size increases, the points fill the region more evenly and provide a clearer approximation of the area under the Weibull density curve.

Table 2: Monte Carlo Estimates for Different Sample Sizes

Sample Size	Estimated Integral
10	0.4000
100	1.2800
1000	0.9960
10000	1.0148

Table 2 summarizes the estimated integral for each sample size. The estimates vary more when the number of simulated points is small, but they become more stable as the sample size increases. Since the Weibull density should integrate to approximately 1 over this interval, the estimates for the larger sample sizes suggest that the Monte Carlo approximation improves as more points are used.

Planning and Prompting GenAI Tools

Plan

Goal:

The goal is to transform the calcium dataset into a tidy format and create a visualization that compares calcium levels over time between the null treatment group and the diet/exercise treatment group.

Needs (Noun and Verbs):

- Nouns: dataset, format, observation, case, measurement, individual, time point, variable, group, time, calcium, data, row, observation
- Verbs: convert, identify, create, ensure, represent

Steps:

1. Import the calcium.csv file into R without modifying the original file.
2. Use `pivot_longer()` to convert the data from wide format to long format.
3. Create a new variable indicating the treatment group (null vs. diet/exercise).
4. Create a time variable that represents Initial, Year 1, Year 2, and Year 3.
5. Clean and rename variables for clarity.
6. Check that each row represents one individual's measurement at one time point.
7. Create a line plot showing calcium levels over time, grouped by treatment group.
8. Add appropriate labels, title, and legend to ensure clarity.
9. Review the plot to confirm it clearly shows differences between groups over time.

Plan-Informed Prompt

I used ChatGPT as my generative AI tool and provided a detailed prompt based on my plan.

Prompt used:

"I have a dataset called calcium.csv that is not tidy.

The dataset contains measurements of calcium levels for 31 women over four time points (Initial, Year 1, Year 2, Year 3). The first four columns represent a null treatment group, and the next four columns represent a diet/exercise treatment group.

I want to:

1. Convert the dataset into a tidy format where each row represents one observation.
2. Create variables for treatment group, time, and calcium level.

3. Ensure that each row represents one individual at one time point.
4. Create a line plot that compares calcium levels over time between the two treatment groups.

Can you explain step-by-step how to tidy this dataset and create the appropriate visualization in R?"

Response from GenAI:

ChatGPT suggested converting the dataset into a long format using a function such as `pivot_longer()`, ensuring that each row represents one observation. It also recommended creating variables for treatment group and time, and using a line plot to visualize calcium levels over time. The response emphasized structuring the data so that it follows tidy data principles, which aligns closely with my planned approach.

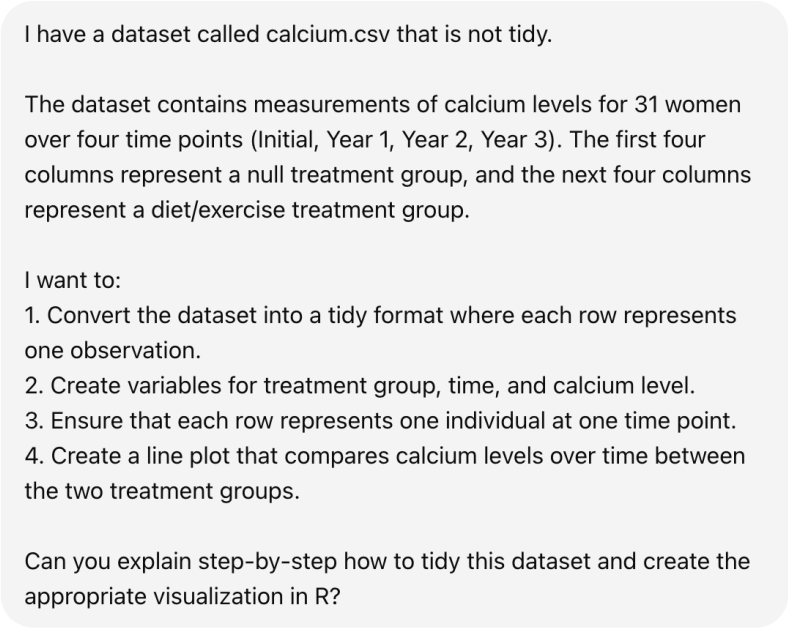

Figure 3: ChatGPT prompt

Step 4: Convert from wide format to long format

Suppose the first four columns are for the null treatment group and the next four columns are for the diet/exercise group. You can reshape the data using `pivot_longer()`.

```r
calcium_long <- calcium |>
  pivot_longer(
    cols = -id,
    names_to = "group_time",
    values_to = "calcium"
  )
```

Now each row contains one calcium measurement, but the group and time are still combined in one variable.

Figure 4: ChatGPT response using a plan-informed prompt (1)

Step 8: Create the line plot

A line plot is appropriate because you want to compare calcium levels over time between two groups.

If you want to show average calcium levels by group over time:

```r
calcium_summary <- calcium_tidy |>
  group_by(group, time) |>
  summarize(
    mean_calcium = mean(calcium, na.rm = TRUE),
    .groups = "drop"
  )

ggplot(calcium_summary, aes(x = time, y = mean_calcium, group = group, color = group)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  labs(
    title = "Average Calcium Levels Over Time by Treatment Group",
    x = "Time",
    y = "Average Calcium Level",
    color = "Treatment Group"
  ) +
  theme_minimal()
```

Figure 5: ChatGPT response using a plan-informed prompt (2)

Figure 3 shows the prompt, and Figure 4-5 show the part of the responses generated using ChatGPT based on the plan.

Generic Prompt Response

I used ChatGPT with a generic prompt that was not based on my plan.

Prompt used: "How can I tidy a dataset and create a plot to show changes over time?"

Response from GenAI: The response suggested converting the dataset into long format using functions such as pivot_longer() and ensuring that each row represents one observation. It also recommended using a line plot to visualize changes over time, with time on the x-axis and the measured variable on the y-axis.

To tidy a dataset, the goal is to ensure that each row represents one observation and each column represents a variable. If the dataset is in a wide format (with multiple columns representing different time points), it can be converted into a long format using functions such as `pivot_longer()` in R. This helps organize the data so that variables like time and measurement are stored in separate columns.

After tidying the data, a line plot is an appropriate way to visualize changes over time. In the plot, time should be placed on the x-axis and the measured variable on the y-axis. If the dataset includes different groups or categories, grouping or color can be used to distinguish between them.

This approach allows for clear visualization of trends over time and makes it easier to compare changes across different groups.

Figure 6: ChatGPT response using a generic prompt

Figure 6 shows the response generated by ChatGPT using a generic prompt.

Comparison

The plan-informed response provided a much more structured and context-specific approach compared to the generic prompt response. In the plan-informed approach, the goal, needs, and steps were clearly defined, which made it easier to understand how to transform the calcium dataset into a tidy format. It also explicitly considered the structure of the dataset, including the presence of two treatment groups and multiple time points.

In contrast, the generic response was more general and did not account for the specific structure of the calcium dataset. While it suggested using pivot_longer() and creating a line plot, it did not clearly explain how to distinguish between treatment groups or how to interpret the data in this particular context.

When comparing the resulting visualizations, the plan-informed approach produces a more meaningful plot because it correctly identifies each observation as a single measurement over time and properly groups the data by treatment type. The generic approach may still produce a plot, but it is more likely to overlook important contextual details, such as the distinction between groups.

Compared to both genAI approaches, my own data wrangling and visualization process demonstrated a clearer understanding of the dataset. I ensured that each row represented a single case and carefully structured the data before plotting. This resulted in a cleaner and more interpretable visualization.

This exercise shows that while genAI tools are helpful, the quality of the output strongly depends on how specific and well-structured the prompt is. A detailed plan leads to more accurate and useful results, while generic prompts tend to produce vague and less context-aware outputs. The plan-informed approach better reflects the structure of the dataset and allows for clearer interpretation of trends across treatment groups.

Reflection

Through this assignment, I learned how to use Quarto to create a structured and reproducible data analysis document. Before this, I had limited experience organizing code, output, and written explanations into a single coherent report, and I initially found it challenging to separate code from narrative. Using Quarto helped me understand how to clearly present results while keeping the document organized and professional.

I also gained a deeper understanding of data wrangling and visualization. In the airport analysis section, I worked with real-world data and practiced tasks such as cleaning messy variables, combining multiple datasets, and creating both tables and graphs. For example, I had to restructure the data and ensure that variables were clearly labeled before plotting. This experience showed me how important it is to organize data properly before creating visualizations.

In the Monte Carlo section, I learned how random sampling can be used to approximate integrals. At first, I found it difficult to understand how random points could estimate an area, but by increasing the number of simulated points, I observed that the estimates became more stable. This helped me better understand the role of sample size and variability in simulation methods.

Finally, the planning and prompting activity helped me see how generative AI tools can be more effective when used with a clear plan. When I used a generic prompt, the response was too general and not very helpful. However, when I used a detailed, plan-based prompt, the response was more structured and relevant to the dataset. This showed me that the quality of AI output depends heavily on how the prompt is written.

Overall, this assignment helped me improve both my technical skills and my ability to communicate data analysis clearly, while also showing me areas where I need more practice, such as handling more complex datasets and writing more precise prompts.

Appendix: GenAI Usage

Tool Used: ChatGPT (OpenAI)

Date of Use: April 16, 2026

Usage 1: Planning and Prompting Section

Prompt (Plan-Informed):

"I have a dataset called calcium.csv that is not tidy.

The dataset contains measurements of calcium levels for 31 women over four time points (Initial, Year 1, Year 2, Year 3). The first four columns represent a null treatment group, and the next four columns represent a diet/exercise treatment group.

I want to:

1. Convert the dataset into a tidy format where each row represents one observation.
2. Create variables for treatment group, time, and calcium level.
3. Ensure that each row represents one individual at one time point.
4. Create a line plot that compares calcium levels over time between the two treatment groups.

Can you explain step-by-step how to tidy this dataset and create the appropriate visualization in R?"

Response (Exact Text):

"To tidy this dataset, you should convert it from a wide format into a long format where each row represents a single observation. You can use pivot_longer() from the tidyr package to reshape the data.

Next, create variables for treatment group and time. The treatment group can be identified based on the original column structure, and the time variable should indicate Initial, Year 1, Year 2, and Year 3.

Ensure that each row represents one individual at one time point. This aligns with tidy data principles.

To visualize the data, create a line plot using ggplot2. Place time on the x-axis and calcium level on the y-axis. Use color or grouping to distinguish between treatment groups. This will allow you to compare trends over time between groups."

Usage 2: Generic Prompt

Prompt:

"How can I tidy a dataset and create a plot to show changes over time?"

Response (Exact Text):

"To tidy a dataset, convert it into long format so that each row represents one observation. Use functions such as pivot_longer() to reshape the data and ensure that variables are clearly labeled.

To visualize changes over time, create a line plot. Place time on the x-axis and the measured variable on the y-axis. Use grouping or color to distinguish between categories if needed. This helps reveal trends over time."

Usage 3: General Quarto / Visualization Help

Prompts:

- "How should I structure a Quarto document for a data analysis assignment that includes multiple sections, visualizations, and appendices?"
- "For a Monte Carlo numerical integration problem, how should I organize my explanation and visualization in a Quarto report?"
- "How can I create a small multiple visualization in R using ggplot2 and combine multiple plots into a single display?"

Response (Exact Text):

"Your Quarto document should be structured with clear sections such as introduction, analysis sections, and appendices. Separate narrative text from code output and ensure that each section includes both explanation and visualizations.

For Monte Carlo integration, explain the process of generating random points and estimating the area under a curve. Use multiple sample sizes to demonstrate how the estimate improves.

For visualization, use ggplot2 to create plots and patchwork to combine multiple plots into a small multiple layout. Ensure all plots have clear labels, titles, and consistent formatting."

Appendix: Code

```r
# Busiest Airport
library(tidyverse)
library(rvest)
library(knitr)
library(kableExtra)
library(patchwork)

url <- "https://en.wikipedia.org/wiki/List_of_busiest_airports_by_passenger_traffic"

tables <- url |>
  read_html() |>
  html_table()

good_tables <- tables |>
  purrr::keep(~ ncol(.x) >= 3)

airports_2025 <- good_tables[[1]] |>
  select(2, last_col()) |>
  rename(Airport = 1, Passengers = 2) |>
  mutate(year = 2025)

airports_2024 <- good_tables[[2]] |>
  select(2, last_col()) |>
  rename(Airport = 1, Passengers = 2) |>
  mutate(year = 2024)

airports_2023 <- good_tables[[3]] |>
  select(2, last_col()) |>
  rename(Airport = 1, Passengers = 2) |>
  mutate(year = 2023)

airports_2022 <- good_tables[[4]] |>
  select(2, last_col()) |>
  rename(Airport = 1, Passengers = 2) |>
  mutate(year = 2022)

airports_2021 <- good_tables[[5]] |>
  select(2, last_col()) |>
  rename(Airport = 1, Passengers = 2) |>
  mutate(year = 2021)
```

```r
airports_2020 <- good_tables[[6]] |>
  select(2, last_col()) |>
  rename(Airport = 1, Passengers = 2) |>
  mutate(year = 2020)

airportData6 <- bind_rows(
  airports_2020,
  airports_2021,
  airports_2022,
  airports_2023,
  airports_2024,
  airports_2025
) |>
  filter(
    str_detect(Airport, "Atlanta") |
      str_detect(Airport, "Frankfurt") |
      str_detect(Airport, "Daxing") |
      str_detect(Airport, "Phoenix") |
      str_detect(Airport, "O'Hare") |
      str_detect(Airport, "Heathrow")
  ) |>
  mutate(
    passengers = readr::parse_number(Passengers),
    IATA = case_when(
      str_detect(Airport, "Atlanta") ~ "ATL",
      str_detect(Airport, "Frankfurt") ~ "FRA",
      str_detect(Airport, "Daxing") ~ "PKX",
      str_detect(Airport, "Phoenix") ~ "PHX",
      str_detect(Airport, "O'Hare") ~ "ORD",
      str_detect(Airport, "Heathrow") ~ "LHR"
    )
  ) |>
  select(year, Airport, IATA, passengers) |>
  arrange(IATA, year)

airportData6_wide <- airportData6 |>
  select(Airport, IATA, year, passengers) |>
  pivot_wider(
    names_from = year,
    values_from = passengers
  ) |>
  arrange(IATA)
```

```r
options(knitr.kable.NA = "")

airportData6_wide |>
  kable(
    caption = "Total Annual Passengers for Six Selected Airports, 2020-2025",
    align = "llcccccc",
    format.args = list(big.mark = ",")
  ) |>
  kable_classic(full_width = FALSE) |>
  footnote(
    general = "Blank cells indicate years in which passenger data were not available in the s
    footnote_as_chunk = TRUE
  )

airportData6 |>
  ggplot(
    aes(
      x = year,
      y = passengers,
      group = IATA,
      linetype = IATA
    )
  ) +
  geom_point() +
  geom_path() +
  theme_bw() +
  labs(
    title = "Total Annual Passengers for Six Selected Airports (2020-2025)",
    x = "Year",
    y = "Total Passengers (in millions)",
    linetype = "Airport"
  ) +
  theme(
    legend.position = "bottom",
    legend.key.width = unit(2, "cm"),
    legend.key.height = unit(0.25, "cm")
  )

# Monte Carlo
set.seed(184)

make_mc_plot <- function(n_points) {
```

```r
mc_points <- tibble(
  x = runif(n_points, 0, 4),
  y = runif(n_points, 0, 1)
)

mc_plot_data <- mc_points |>
  mutate(
    curve_y = dweibull(x, shape = 1.5, scale = 1),
    location = if_else(y <= curve_y, "On/Below Curve", "Above Curve")
  )

mc_estimate <- mc_plot_data |>
  summarize(
    estimate = mean(location == "On/Below Curve") * 4 * 1
  ) |>
  pull(estimate)

p <- ggplot(mc_plot_data, aes(x = x, y = y)) +
  geom_point(
    aes(color = location),
    alpha = 0.6,
    size = 1.2
  ) +
  stat_function(
    aes(group = 1),
    fun = dweibull,
    args = list(shape = 1.5, scale = 1),
    xlim = c(0, 4),
    color = "black",
    linewidth = 0.8
  ) +
  scale_color_manual(
    values = c(
      "On/Below Curve" = "deepskyblue3",
      "Above Curve" = "tomato2"
    )
  ) +
  scale_x_continuous(limits = c(0, 4), expand = expansion()) +
  scale_y_continuous(limits = c(0, 1), expand = expansion()) +
  labs(
    title = NULL,
    subtitle = paste("n =", n_points),
```

```r
      caption = paste("Estimated Integral =", round(mc_estimate, 4)),
      x = "x",
      y = "y"
    ) +
    theme_bw() +
    theme(
      legend.position = "none"
    )

  return(p)
}

mc10 <- make_mc_plot(10)
mc100 <- make_mc_plot(100)
mc1000 <- make_mc_plot(1000)
mc10000 <- make_mc_plot(10000)

mc_results <- tibble(
  `Sample Size` = c(10, 100, 1000, 10000),
  `Estimated Integral` = c(
    0.4, 1.28, 0.996, 1.0148
  )
)

(mc10 | mc100) /
(mc1000 | mc10000) +
  plot_annotation(
    title = "Monte Carlo Integration for a Weibull Density"
  )

mc_results |>
  kable(
    caption = "Monte Carlo Estimates for Different Sample Sizes",
    col.names = c("Sample Size", "Estimated Integral"),
    digits = 4
  ) |>
  kable_classic(full_width = FALSE)
```